GUIA VISUAL 033

APT E O MODELO DEBIAN

A pergunta certa não é "onde eu baixo o nmap" — é "isso já está no repositório?".

033
GUIAS LINUX

UMA LINHA DO `SOURCES.LIST`, CAMPO A CAMPO

```
deb  http://http.kali.org/kali  kali-rolling  main contrib non-free non-free-firmware
```

TIPO
`deb` = binário pronto; `deb-src` = código-fonte

URI
Endereço do espelho. O `http.kali.org` é um **redirecionador**, não um servidor: manda para o mirror mais próximo.

SUÍTE
A linha de distribuição — `kali-rolling`, `kali-last-snapshot`, `kali-experimental`

COMPONENTES
Seções por **licença**, não por tema: livre; livre que depende de não livre; restritivo; e firmware.

O APT **concatena os campos numa URL**: `URI/dists/SUÍTE/COMPONENTE/binary-amd64/Packages.gz` — por isso o erro de origem sai como um **404 mostrando a URL inteira**, e o diagnóstico é só ler qual campo está errado. O formato **deb822** (`.sources`) diz o mesmo em chave-valor e ainda amarra a origem à sua chave com `Signed-By:`.

COMO O APT CONFIA: UMA CHAVE ESTENDE GARANTIA A MILHARES DE ARQUIVOS

chave GPG (kali-archive-keyring, local) → valida a assinatura → **InRelease** (assinado; traz o hash dos índices) → valida por hash → **Packages.gz** (índice; traz o hash de cada .deb) → valida por hash → **pacote.deb** (o arquivo que será instalado)

- `NO_PUBKEY <ID>` — a chave que assinou o repositório não está instalada.
- `EXPKEYSIG` — a chave venceu; atualize o pacote de chaveiro (o Kali rotaciona a dele).
- `não é válido ainda` — relógio atrasado (VM suspensa, live USB). `timedatectl` resolve.

DUAS CAMADAS: DPKG EMBAIXO, APT EM CIMA

- ✓ **dpkg é o operário:** desempacota, roda scripts e registra — mas **nunca sai à rede**. Falta dependência, ele aborta.
- ✓ **APT é o planejador:** lê os índices, resolve o grafo, escolhe a versão, baixa e chama o dpkg na ordem.
- ✓ "`dpkg -i` deu erro de dependência" se resolve **subindo uma camada**: `apt install -f`.
- ✓ `apt update` **não instala nada**: sincroniza índices em `/var/lib/apt/lists/` e confere assinaturas.
- ✓ `apt policy nmap` revela a decisão: **100** = instalado, **500** = origem normal, **990** = release alvo. Vence a maior.

ROLLING: O KALI NÃO TEM "VERSÃO"

- ✓ Debian **stable** congela e só recebe correção de segurança por anos. O Kali deriva de **testing** e recebe pacote continuamente.
- ✓ As imagens numeradas (`2025.3`) são **snapshots de instalação**: a 2025.1 atualizada hoje é idêntica à 2025.3.
- ✓ A contrapartida: atualizar é operação com risco real — versão principal muda, opção some, config é substituída.
- ✓ `kali-last-snapshot` congela o ambiente (turma, laboratório reproduzível); voltar é editar uma linha.
- ✓ Índice de rolling **vence em dias**: o `404` ao instalar num Kali parado há um mês é índice velho, não pacote removido.

Nunca misture repositórios de distribuições diferentes. Adicionar ao Kali um PPA, o Ubuntu ou o Debian stable "porque o pacote não tem aqui" arrasta a `libc6` daquela origem — e no meio da substituição **nada mais executa, inclusive o próprio apt**, o que impede o desfazer. A única origem legítima do Kali é a do Kali; o resto se resolve compilando, com `pipx`, contêiner ou AppImage. E **jamais contorne um erro de GPG com `[trusted=yes]`**: isso não conserta o elo, desliga a cadeia inteira — em silêncio e para sempre.

RESUMO RÁPIDO

- ✓ Repositório = diretório HTTP: `.deb` em `pool/`, índices que os descrevem e um release **assinado**.
- ✓ Uma linha do `sources.list`: `deb URI SUÍTE COMPONENTES` — o erro de origem sai como **404 mostrando a URL inteira**.
- ✓ **dpkg** instala o que você entrega; **APT** resolve e baixa. `apt update` só sincroniza índices.
- ✓ Kali é **rolling**: sem versão, sem upgrade de distro; `kali-last-snapshot` congela o ambiente.
- ✓ Nunca misture repositórios de distribuições diferentes; nunca contorne um erro de GPG.

NO KALI

Uma linha só deve estar no `sources.list`: `deb http://http.kali.org/kali kali-rolling main contrib`